

# Taller IV: El Algoritmo PageRank de Google

Reinaldo Kevin Díaz Parra C.I V-27979417

September 5, 2026

## 1 Introducción

El crecimiento acelerado de la World Wide Web durante la década de 1990 planteó un desafío computacional crítico: cómo clasificar y recuperar información relevante dentro de una red masiva, descentralizada y no estructurada. Los motores de búsqueda iniciales se basaban principalmente en la coincidencia de texto e índices lexicográficos, un enfoque vulnerable a la manipulación de contenido y con poca capacidad para determinar la autoridad real de un documento. En este contexto, Larry Page y Sergey Brin formularon el algoritmo PageRank, proponiendo que la importancia de una página web no está condicionada únicamente por el volumen de enlaces que recibe, sino fundamentalmente por la jerarquía y relevancia intrínseca de las páginas que la referencian.

Desde una perspectiva analítica, PageRank modela la navegación de un usuario hipotético a través de un grafo dirigido  $G = (V, E)$ , donde cada vértice  $v \in V$  representa una página web y cada arista dirigida  $(u, v) \in E$  denota un hipervínculo entre ellas. Esta dinámica se formaliza rigurosamente mediante la teoría de cadenas de Markov de tiempo discreto. La navegación se describe a través de una matriz de transición  $M \in \mathbb{R}^{n \times n}$ , cuyas columnas cuantifican la dispersión equitativa del prestigio de un nodo  $j$  entre sus  $C_j$  enlaces salientes.

Sin embargo, las redes reales exhiben patologías estructurales que quiebran la ergodicidad del sistema, tales como nodos sumidero (dangling nodes, donde  $C_j = 0$ ) y ciclos absorbentes cerrados que atrapan la probabilidad. Para superar estas restricciones, el modelo incorpora un factor de amortiguamiento  $d$  (típicamente  $d = 0.85$ ), dando origen a la Matriz de Google  $G$ :

$$G = dM + \frac{1-d}{n}J$$

Esta transformación perturba la estructura original convirtiendo a  $G$  en una matriz estocástica irreducible y primitiva con entradas estrictamente positivas. Bajo el marco del Teorema de Perron-Frobenius, esta condición asegura la existencia, unicidad y positividad de un autovector estacionario  $v$  asociado al autovalor dominante  $\lambda = 1$ , el cual define el vector definitivo de rangos de las páginas.

Dado que la dimensión de la web imposibilita resolver el sistema mediante determinantes o la inversión directa de matrices, el cálculo se aborda a través de métodos de álgebra lineal numérica, específicamente el Método de las Potencias. Este esquema aproxima iterativamente el autovector dominante mediante la relación  $v^{(k+1)} = Gv^{(k)}$  hasta satisfacer una tolerancia prefijada.

El presente informe expone el modelado matricial de diversas topologías de red, analiza el impacto de ciclos y nodos sin salida en la distribución asintótica del flujo, detalla la implementación computacional del método numérico en Python y evalúa cómo la topología de interconexión determina el ranking global sobre un simple conteo de enlaces entrantes.

## 2 Fundamentos Teóricos

El algoritmo PageRank descansa en la intersección de tres áreas fundamentales de la matemática aplicada: la teoría de grafos, las cadenas de Markov de tiempo discreto y el análisis espectral en álgebra lineal numérica.

## 2.1 Representación Matricial de Grafos Orientados

Sea  $G = (V, E)$  un grafo dirigido que modela una red compuesta por  $n$  páginas web, donde el conjunto de vértices  $V = \{0, 1, \dots, n-1\}$  representa los sitios web y el conjunto de aristas dirigidas  $E \subseteq V \times V$  describe los hipervínculos existentes entre ellos. La conectividad del sistema se codifica algebraicamente mediante la matriz de adyacencia  $A \in \{0, 1\}^{n \times n}$ , definida formalmente como:

$$A_{ij} = \begin{cases} 1 & \text{si existe un hipervínculo saliente del nodo } i \text{ hacia el nodo } j, \\ 0 & \text{en caso contrario.} \end{cases} \quad (1)$$

El número total de enlaces salientes que emergen de una página  $j$ , denominado grado de salida  $C_j$ , se obtiene sumando los elementos correspondientes a su fila en  $A$ :

$$C_j = \sum_{k=0}^{n-1} A_{jk} \quad (2)$$

A partir de la topología de la red, se modela el flujo de navegación mediante la matriz de transición de probabilidad  $M \in \mathbb{R}^{n \times n}$ . En esta formulación, las columnas parametrizan la página de origen y las filas indexan la página de destino:

$$M_{ij} = \begin{cases} \frac{1}{C_j} & \text{si } (j, i) \in E, \\ 0 & \text{en otro caso.} \end{cases} \quad (3)$$

Bajo esta convención, cada nodo distribuye su autoridad equitativamente entre sus páginas adyacentes de salida. Cuando todos los vértices del grafo poseen al menos una arista saliente ( $C_j > 0, \forall j$ ), la matriz  $M$  es estocástica por columnas, verificando:

$$\sum_{i=0}^{n-1} M_{ij} = 1, \quad \forall j \in \{0, 1, \dots, n-1\} \quad (4)$$

## 2.2 Tratamiento de Nodos Sumidero (*Dangling Nodes*)

En grafos orientados reales existen páginas carentes de hipervínculos salientes ( $C_j = 0$ ), denominadas nodos sumidero o *dangling nodes*. En términos matriciales, un sumidero produce una columna nula en  $M$  ( $\sum_{i=0}^{n-1} M_{ij} = 0$ ), lo que destruye la conservación de la medida de probabilidad en la cadena de Markov subyacente.

Para restaurar la propiedad estocástica del operador, la columna correspondiente a cualquier vértice con  $C_j = 0$  se reemplaza por una distribución uniforme discreta  $w \in \mathbb{R}^n$ , con componentes  $w_i = \frac{1}{n}$ . La matriz de transición corregida  $M^* \in \mathbb{R}^{n \times n}$  se define como:

$$M_{ij}^* = \begin{cases} \frac{1}{C_j} & \text{si } C_j > 0 \text{ y } A_{ji} = 1, \\ \frac{1}{n} & \text{si } C_j = 0, \\ 0 & \text{en otro caso.} \end{cases} \quad (5)$$

Esta regularización asegura que  $\sum_{i=0}^{n-1} M_{ij}^* = 1$  para toda columna  $j \in \{0, \dots, n-1\}$ .

## 2.3 El Factor de Amortiguamiento y la Matriz de Google $G$

Aun cuando  $M^*$  es estocástica, los grafos pueden albergar subestructuras desconectadas o componentes fuertemente conexas sin salidas que actúan como trampas absorbentes. Con el fin de dotar al sistema de ergodicidad, se introduce el factor de amortiguamiento  $d \in (0, 1)$  (típicamente  $d = 0.85$ ), formulando la Matriz de Google  $G \in \mathbb{R}^{n \times n}$ :

$$G = dM^* + \frac{1-d}{n}J \quad (6)$$

donde  $J \in \mathbb{R}^{n \times n}$  denota la matriz unitaria cuyas entradas son todas idénticamente iguales a 1. Físicamente, este operador descompone la dinámica en dos procesos estocásticos:

- **Navegación determinística ( $d$ ):** Con probabilidad  $d$ , el usuario sigue de manera uniforme los enlaces presentes en la estructura del grafo.
- **Teletransportación aleatoria ( $1-d$ ):** Con probabilidad  $1-d$ , el usuario abandona la secuencia de navegación y accede directamente a cualquier página de la red con probabilidad equiprobable  $\frac{1}{n}$ .

Dado que  $\frac{1-d}{n} > 0$  y  $J_{ij} = 1$ , se cumple que  $G_{ij} > 0$  para todo par de índices  $(i, j)$ . Por consiguiente,  $G$  es una matriz estrictamente positiva y estocástica por columnas.

## 2.4 Fundamento Espectral y Teorema de Perron-Frobenius

Las propiedades asintóticas de la cadena de Markov se desprenden de manera directa del **Teorema de Perron-Frobenius**:

1. Como  $G$  es estrictamente positiva ( $G_{ij} > 0$ ), su radio espectral es unitario,  $\rho(G) = 1$ , estableciendo como autovalor dominante a  $\lambda_1 = 1$ .
2. La multiplicidad algebraica y geométrica de  $\lambda_1 = 1$  es estrictamente igual a 1, garantizando que el subespacio invariante asociado es unidimensional.
3. Existe un único autovector a derecha  $v \in \mathbb{R}^n$ , con todas sus componentes estrictamente positivas ( $v_i > 0, \forall i$ ), que satisface la ecuación de punto fijo:

$$Gv = v, \quad \text{sujeto a} \quad \sum_{i=0}^{n-1} v_i = 1 \quad (7)$$

4. Para cualquier otro autovalor  $\lambda_k$  ( $k \geq 2$ ) en el espectro de  $G$ , se satisface la cota de magnitud  $|\lambda_k| \leq d = 0.85$ , asegurando una separación espectral mínima no nula  $\gamma \geq 1 - d$ .

## 2.5 El Método de las Potencias y Tasa de Convergencia

Dado que la cardinalidad de la red en aplicaciones prácticas vuelve intratable el cálculo mediante polinomios característicos o descomposiciones matriciales directas, se recurre a esquemas iterativos de álgebra lineal numérica. El autovector estacionario se aproxima mediante el **Método de las Potencias**:

$$v^{(k+1)} = Gv^{(k)} \quad (8)$$

Fijando una distribución inicial normalizada  $v^{(0)} = [\frac{1}{n}, \frac{1}{n}, \dots, \frac{1}{n}]^T$ , el vector de estado se descompone en la base canónica de autovectores  $\{u_1, u_2, \dots, u_n\}$  de  $G$ :

$$v^{(0)} = u_1 + \sum_{k=2}^n c_k u_k \quad (9)$$

Al aplicar inductivamente el operador  $G$  tras  $m$  iteraciones sucesivas:

$$v^{(m)} = G^m v^{(0)} = (\lambda_1)^m u_1 + \sum_{k=2}^n c_k (\lambda_k)^m u_k = u_1 + \sum_{k=2}^n c_k (\lambda_k)^m u_k \quad (10)$$

Puesto que  $|\lambda_k| \leq d < 1$  para todo  $k \geq 2$ , se tiene que  $\lim_{m \rightarrow \infty} (\lambda_k)^m = 0$ . En consecuencia, el error respecto a la distribución invariante decrece a una razón geométrica acotada por  $d$ :

$$\|v^{(k+1)} - v\| \leq d \|v^{(k)} - v\| \quad (11)$$

El algoritmo concluye cuando la norma de la diferencia entre dos iteraciones continuas satisface la cota de tolerancia requerida:

$$\|v^{(k+1)} - v^{(k)}\|_1 < \varepsilon = 10^{-8} \quad (12)$$

### 3 Desarrollo Experimental y Metodología Computacional

En esta sección se detalla el diseño topológico de las redes evaluadas, la formulación explícita de sus matrices de adyacencia y la arquitectura computacional desarrollada en Python para la resolución numérica del PageRank mediante el Método de las Potencias.

#### 3.1 Diseño de la Red Propuesta ( $n = 6$ )

Siguiendo los requerimientos, se diseñó una red dirigida conformada por  $n = 6$  vértices (indexados del 0 al 5) que incorpora simultáneamente una estructura cíclica y un nodo sumidero ( $C_j = 0$ ):

- **Subestructura Cíclica:** Conformada por el anillo cerrado  $0 \rightarrow 1 \rightarrow 2 \rightarrow 0$ , el cual actúa como un lazo recurrente de realimentación de probabilidad.
- **Camino de Dispersión:** A partir del nodo 2 emerge una bifurcación hacia una trayectoria lineal no orientada al ciclo:  $2 \rightarrow 3$ ,  $3 \rightarrow 4$  y  $4 \rightarrow 5$ .
- **Nodo Sumidero:** El vértice 5 carece de aristas salientes ( $C_5 = 0$ ), constituyendo un punto de acumulación estocástica que pone a prueba el mecanismo de teletransportación uniforme.

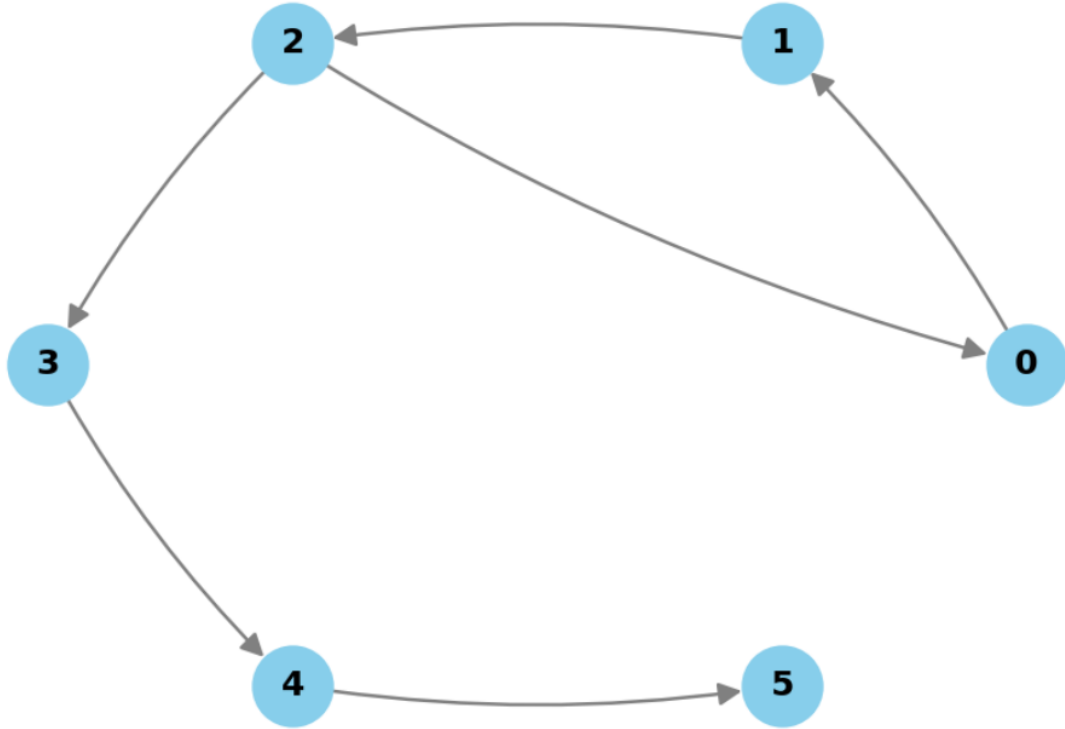


Figure 1: Topología del grafo propuesto de 6 nodos: se evidencia el ciclo  $\{0, 1, 2\}$ , la ramificación unidireccional  $\{3, 4\}$  y el nodo sumidero terminal 5.

La matriz de adyacencia resultante  $A_{\text{prop}} \in \{0, 1\}^{6 \times 6}$  se expresa como:

$$A_{\text{prop}} = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad (13)$$

### 3.2 Topologías de los Casos de Prueba (Grafos 1 al 5)

Se analizaron las configuraciones suministradas con el fin de evaluar la robustez del operador ante diferentes patologías estructurales:

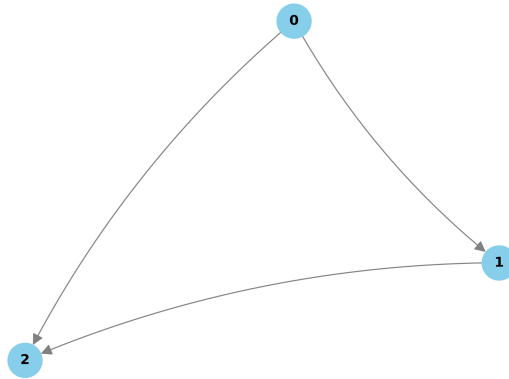


Figure 2: Grafo 1 (3 nodos con sumidero terminal)

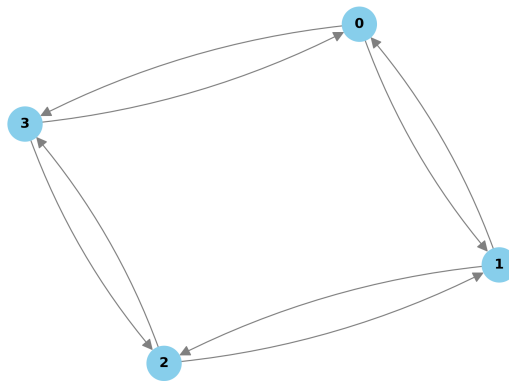


Figure 3: Grafo 2 (4 nodos con simetría bidireccional).

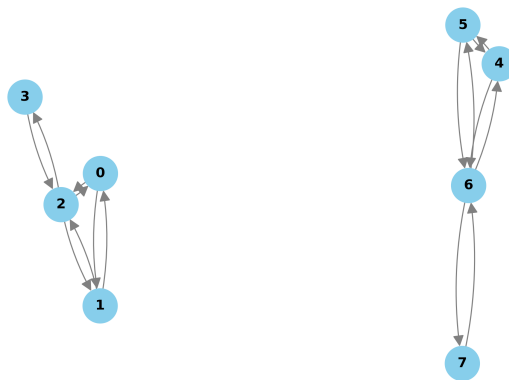


Figure 4: Grafo 3: Sistema disjunto de 8 nodos particionado en dos subgrafos fuertemente conexos e independientes ( $\{0, 1, 2, 3\}$  y  $\{4, 5, 6, 7\}$ ).

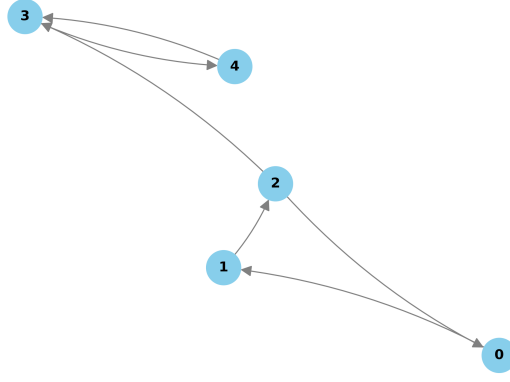


Figure 5: Grafo 4 (5 nodos con interconexión cíclica múltiple)

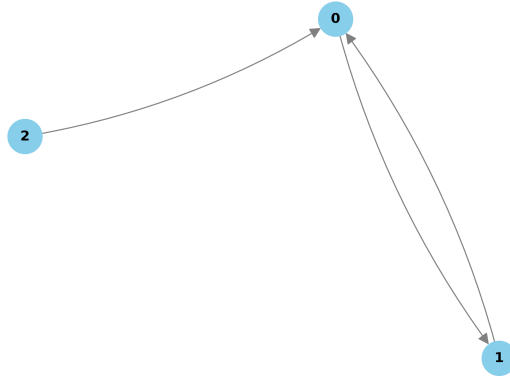


Figure 6: Grafo 5 (3 nodos con fuente pura).

Las matrices de adyacencia correspondientes a cada topología son:

- **Grafo 1** ( $n = 3$ ): Contiene enlaces  $0 \rightarrow 1$ ,  $0 \rightarrow 2$  y  $1 \rightarrow 2$ . El nodo 2 es un sumidero.

$$A_1 = \begin{pmatrix} 0 & 1 & 1 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix} \quad (14)$$

- **Grafo 2** ( $n = 4$ ): Estructura cíclica simétrica bidireccional ( $0 \rightleftharpoons 1 \rightleftharpoons 2 \rightleftharpoons 3 \rightleftharpoons 0$ ).

$$A_2 = \begin{pmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{pmatrix} \quad (15)$$

- **Grafo 3** ( $n = 8$ ): Dos componentes disyuntas sin aristas intermedias.

$$A_3 = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix} \quad (16)$$

- **Grafo 4** ( $n = 5$ ): Ciclo base  $\{0 \rightarrow 1 \rightarrow 2 \rightarrow 0\}$  conectado a un dipolo recíproco  $\{3 \rightleftharpoons 4\}$  mediante la arista de enlace  $2 \rightarrow 3$ .

$$A_4 = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix} \quad (17)$$

- **Grafo 5** ( $n = 3$ ): Enlace asimétrico  $2 \rightarrow 0$  que alimenta un dipolo recíproco  $0 \rightleftharpoons 1$ . El nodo 2 es una fuente pura (grado de entrada nulo).

$$A_5 = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 0 & 0 \end{pmatrix} \quad (18)$$

### 3.3 Implementación Computacional del Algoritmo

El código desarrollado en Python se estructuró para operar de forma matricial vectorizada haciendo uso de la biblioteca `numpy`:

1. **Cálculo de Grados de Salida:** Se computa el vector  $C \in \mathbb{R}^n$  mediante sumas por fila en la matriz de adyacencia:  $C = \sum_j A_{ij}$ .
2. **Construcción de la Matriz de Transición  $M$ :** Se inicializa una matriz nula de orden  $n \times n$ . Para cada columna  $j$ , si  $C_j > 0$ , se asigna  $M_{:,j} = A_{j,:}^T / C_j$ . Si  $C_j = 0$ , se aplica la regularización estocástica asignando  $M_{:,j} = 1/n$ .
3. **Ensamblaje de la Matriz de Google  $G$ :** Con  $d = 0.85$  y  $J = \mathbf{1}_{n \times n}$ , se evalúa directamente  $G = dM + \frac{1-d}{n}J$ .
4. **Método de las Potencias:** Se inicializa el vector de rangos equiprobable  $v^{(0)} = \frac{1}{n}\mathbf{1}_n$ . En cada iteración  $k$ , se evalúa  $v^{(k+1)} = Gv^{(k)}$  y se verifica la condición de Cauchy en norma  $L_1$ :  $\|v^{(k+1)} - v^{(k)}\|_1 < 10^{-8}$ .

### 3.4 Implementación del Script en Python

A continuación, se presenta el código fuente íntegro desarrollado para ejecutar en Google Colab. El script define las matrices de adyacencia correspondientes a los casos de prueba y a la red propuesta, construye la matriz estocástica regularizada  $M$ , calcula la matriz de Google  $G$  y aplica el Método de las Potencias con una tolerancia de parada  $\varepsilon = 10^{-8}$ .

```

1 import numpy as np
2
3
4 # 1. DEFINICION DE LAS MATRICES DE ADYACENCIA
5 # Convención: A[i][j] = 1 significa que hay un enlace del nodo 'i' al nodo 'j'
6
7
8 # RED PROPUESTA (6 Nodos)
9 # Tiene un ciclo (0->1->2->0) y un nodo sumidero (5 no tiene salidas)
10 A_propuesta = np.array([
11     [0, 1, 0, 0, 0, 0], # Nodo 0 apunta a 1
12     [0, 0, 1, 0, 0, 0], # Nodo 1 apunta a 2
13     [1, 0, 0, 1, 0, 0], # Nodo 2 apunta a 0 y 3 (Cierra ciclo 0-1-2)
14     [0, 0, 0, 0, 1, 0], # Nodo 3 apunta a 4
15     [0, 0, 0, 0, 0, 1], # Nodo 4 apunta a 5
16     [0, 0, 0, 0, 0, 0] # Nodo 5 es sumidero (C_5 = 0)
17 ])
18
19 # GRAFO 1
20 # Nodos: 0, 1, 2. Enlaces: 0->1, 0->2, 1->2.
21 # El nodo 2 es un sumidero.
22 A_grafo_1 = np.array([

```

```

23     [0, 1, 1],
24     [0, 0, 1],
25     [0, 0, 0]
26 ])
27
28 # GRAFO 2
29 # Nodos: 0, 1, 2, 3.
30 # Es un grafo fuertemente conectado en forma de anillo bidireccional.
31 # Enlaces: 0<->1, 1<->2, 2<->3, 3<->0.
32 A_grafo_2 = np.array([
33     [0, 1, 0, 1],
34     [1, 0, 1, 0],
35     [0, 1, 0, 1],
36     [1, 0, 1, 0]
37 ])
38
39 # GRAFO 3
40 # Nodos: 0 al 7.
41 # Son dos subgrafos completamente desconectados (0,1,2,3 y 4,5,6,7).
42 A_grafo_3 = np.array([
43     [0, 1, 1, 0, 0, 0, 0, 0], # Nodo 0
44     [1, 0, 1, 0, 0, 0, 0, 0], # Nodo 1
45     [1, 1, 0, 1, 0, 0, 0, 0], # Nodo 2
46     [0, 0, 1, 0, 0, 0, 0, 0], # Nodo 3
47     [0, 0, 0, 0, 0, 1, 1, 0], # Nodo 4
48     [0, 0, 0, 0, 1, 0, 1, 0], # Nodo 5
49     [0, 0, 0, 0, 1, 1, 0, 1], # Nodo 6
50     [0, 0, 0, 0, 0, 0, 1, 0] # Nodo 7
51 ])
52
53 # GRAFO 4
54 # Nodos: 0, 1, 2, 3, 4.
55 # Enlaces: 0->1, 2->0, 1->2, 2->3, y bidireccional entre 3<->4.
56 A_grafo_4 = np.array([
57     [0, 1, 0, 0, 0], # Nodo 0 apunta a 1
58     [0, 0, 1, 0, 0], # Nodo 1 apunta a 2
59     [1, 0, 0, 1, 0], # Nodo 2 apunta a 0 y 3
60     [0, 0, 0, 0, 1], # Nodo 3 apunta a 4
61     [0, 0, 0, 1, 0] # Nodo 4 apunta a 3
62 ])
63
64 # GRAFO 5
65 # Nodos: 0, 1, 2.
66 # Enlaces: 2->0, y bidireccional 0<->1.
67 A_grafo_5 = np.array([
68     [0, 1, 0], # Nodo 0 -> 1
69     [1, 0, 0], # Nodo 1 -> 0
70     [1, 0, 0] # Nodo 2 -> 0
71 ])
72
73
74
75 # 2. IMPLEMENTACION DEL ALGORITMO PAGERANK
76
77
78 def calcular_pagerank(A, nombre_grafo, d=0.85, tol=1e-8, max_iter=1000):
79     """
80     Construye la matriz de Google y aplica el metodo de las potencias.
81     """
82     print(f"Analisis {nombre_grafo}")
83
84     A = np.array(A, dtype=float)
85     n = A.shape[0]
86     print(f"1. Matriz de Adyacencia A ({n}x{n}):\n{A}\n")
87
88     # Calcular el grado de salida (C_j) de cada nodo sumando las filas
89     C = np.sum(A, axis=1)
90
91     # Construccion de la Matriz de Transicion M
92     # M_ij = 1 / C_j si hay enlace de j a i, de lo contrario 0
93     M = np.zeros((n, n))

```



```

94     for j in range(n): # j itera sobre las paginas de ORIGEN (columnas de M)
95         if C[j] > 0:
96             M[:, j] = A[j, :] / C[j] # Transposicion y division
97         else:
98             # MANEJO DE NODOS SUMIDERO (C_j = 0)
99             # Para evitar "sumideros", repartimos la probabilidad entre todos los
            nodos por igual
100             M[:, j] = 1.0 / n
101
102     print(f"2. Matriz de Transicion M:\n{np.round(M, 4)}\n")
103
104     # Construccion de la Matriz de Google G
105     #  $G = d * M + ((1 - d) / n) * J$ 
106     J = np.ones((n, n))
107     G = d * M + ((1 - d) / n) * J
108
109     print(f"3. Matriz de Google G (Factor d={d}):\n{np.round(G, 4)}\n")
110
111     # Metodo de las Potencias
112     #  $v^{(k+1)} = G * v^{(k)}$ 
113     v = np.ones(n) / n # Vector inicial con distribución uniforme
114
115     iteraciones = 0
116     for i in range(max_iter):
117         v_siguiente = np.dot(G, v)
118
119         # Evaluar convergencia usando la diferencia entre iteraciones
120         diferencia = np.linalg.norm(v_siguiente - v, ord=1)
121         v = v_siguiente
122
123         if diferencia < tol:
124             iteraciones = i + 1
125             break
126
127     print(f"4. Convergencia del Metodo de las Potencias:")
128     if iteraciones > 0:
129         print(f"    El algoritmo convergio en {iteraciones} iteraciones (Tolerancia: {
            tol}).")
130     else:
131         print(f"    Advertencia ! No convergio despues de {max_iter} iteraciones.")
132
133     print("\n    Vector PageRank Final (v):")
134     for idx, rank in enumerate(v):
135         print(f"    Nodo {idx}: {rank:.6f}")
136
137     return v, M, G
138
139 # 3. EJECUCION DE TODOS LOS CASOS DE PRUEBA
140
141 # Lista de todos los grafos a evaluar
142 grafos_a_evaluar = [
143     (A_propuesta, "Red Propuesta (6 Nodos)"),
144     (A_grafo_1, "Grafo 1 "),
145     (A_grafo_2, "Grafo 2 "),
146     (A_grafo_3, "Grafo 3 "),
147     (A_grafo_4, "Grafo 4 "),
148     (A_grafo_5, "Grafo 5 ")
149 ]
150
151 # Ejecutar el algoritmo para cada matriz
152 for matriz, nombre in grafos_a_evaluar:
153     calcular_pagerank(matriz, nombre, d=0.85, tol=1e-8)

```

Listing 1: Implementación del algoritmo PageRank y Método de las Potencias

## 4 Análisis de Resultados y Discusión

En esta sección se exponen los vectores estacionarios de PageRank obtenidos computacionalmente para cada una de las redes evaluadas mediante el Método de las Potencias con una tolerancia de parada  $\varepsilon = 10^{-8}$  y un factor de amortiguamiento estándar  $d = 0.85$ . Se analiza el comportamiento de la convergencia numérica y la respuesta del modelo ante las diferentes patologías estructurales.

### 4.1 Resultados Numéricos y Desempeño del Algoritmo

El cuadro 1 resume la distribución de probabilidad estacionaria  $v \in \mathbb{R}^n$ , la cantidad de iteraciones requeridas para alcanzar la tolerancia prefijada y la norma de discrepancia final para cada configuración analizada.

Table 1: Resultados numéricos del algoritmo PageRank ( $d = 0.85$ ,  $\varepsilon = 10^{-8}$ ).

| Grafo Evaluado            | Iteraciones | Error ( $\ v^{(k+1)} - v^{(k)}\ _1$ ) | Vector PageRank Resultante ( $v$ )                                                                                         |
|---------------------------|-------------|---------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Red Propuesta ( $n = 6$ ) | 49          | $9.08 \times 10^{-9}$                 | $v_0 = 0.1584, v_1 = 0.1596, v_2 = 0.1607,$<br>$v_3 = 0.0933, v_4 = 0.1043, v_5 = 0.3237$                                  |
| Grafo 1 ( $n = 3$ )       | 30          | $6.25 \times 10^{-9}$                 | $v_0 = 0.1865, v_1 = 0.2658, v_2 = 0.5477$                                                                                 |
| Grafo 2 ( $n = 4$ )       | 2           | 0.00                                  | $v_0 = 0.2500, v_1 = 0.2500,$<br>$v_2 = 0.2500, v_3 = 0.2500$                                                              |
| Grafo 3 ( $n = 8$ )       | 41          | $8.11 \times 10^{-9}$                 | $v_0 = 0.1345, v_1 = 0.1345, v_2 = 0.1706,$<br>$v_3 = 0.0604, v_4 = 0.1345, v_5 = 0.1345,$<br>$v_6 = 0.1706, v_7 = 0.0604$ |
| Grafo 4 ( $n = 5$ )       | 68          | $8.95 \times 10^{-9}$                 | $v_0 = 0.0465, v_1 = 0.0660, v_2 = 0.0847,$<br>$v_3 = 0.4014, v_4 = 0.4014$                                                |
| Grafo 5 ( $n = 3$ )       | 47          | $9.82 \times 10^{-9}$                 | $v_0 = 0.4792, v_1 = 0.4573, v_2 = 0.0635$                                                                                 |

### 4.2 Análisis Estructural y Fenomenológico de los Grafos

#### 4.2.1 Red Propuesta ( $n = 6$ ): Dinámica de Ciclo y Sumidero Terminal

En la red propuesta, los nodos del lazo cíclico  $\{0, 1, 2\}$  exhiben un nivel de relevancia balanceado y elevado ( $v_0 \approx 0.1584, v_1 \approx 0.1596, v_2 \approx 0.1607$ ) debido a la recirculación cerrada del flujo estocástico. El nodo 2 transfiere la mitad de su probabilidad saliente al nodo 3 ( $C_2 = 2$ ), dispersando masa hacia la rama lineal  $\{3 \rightarrow 4 \rightarrow 5\}$ . Conforme el flujo se propaga, el nodo sumidero 5 ( $C_5 = 0$ ) acumula la mayor fracción de PageRank ( $v_5 \approx 0.3237$ ). Este comportamiento valida el diseño: un nodo sin enlaces salientes actúa como un pozo de potencial donde la probabilidad confluye antes de redistribuirse uniformemente mediante la teletransportación estocástica impuesta por  $M^*$ .

#### 4.2.2 Grafo 1: Gradiente de Flujo Acíclico

El Grafo 1 es un grafo acíclico dirigido (DAG) donde la conectividad sigue una estructura jerárquica estricta. El nodo 0 dispersa todo su peso hacia 1 y 2, el nodo 1 canaliza su flujo recibido hacia 2, y el vértice 2 absorbe la convergencia de ambas ramas sin emitir conexiones salientes. Esta configuración genera una estratificación creciente monótona ( $v_0 < v_1 < v_2$ ), consolidando al nodo sumidero 2 con el 54.77% de la masa probabilística total.

#### 4.2.3 Grafo 2: Simetría Espectral y Ergodicidad Inmediata

El Grafo 2 representa un anillo bidireccional regular donde cada nodo posee un grado de salida idéntico ( $C_j = 2$ ) y un grado de entrada simétrico. Debido a la regularidad de la matriz de transición  $M$ , la distribución uniforme inicial  $v^{(0)} = [0.25, 0.25, 0.25, 0.25]^T$  coincide exactamente con el autovector invariante asociado a  $\lambda_1 = 1$ . En consecuencia, el algoritmo converge de forma trivial en dos iteraciones, ilustrando que en redes regulares no transitivas, la teletransportación no perturba el equilibrio topológico inicial.

#### 4.2.4 Grafo 3: Preservación de la Ergodicidad en Redes Desconectadas

El Grafo 3 está dividido en dos componentes disyuntas no comunicadas entre sí:  $\{0, 1, 2, 3\}$  y  $\{4, 5, 6, 7\}$ . En una cadena de Markov convencional sin perturbación ( $d = 1$ ), el autovalor  $\lambda = 1$  presentaría multiplicidad geométrica 2, impidiendo la existencia de una distribución estacionaria única. La incorporación de la matriz de teletransportación  $\frac{1-d}{n}J$  convierte a la matriz de Google  $G$  en irreducible y aperiódica, garantizando convergencia en 41 iteraciones a un perfil simétrico perfecto entre ambas componentes ( $v_i = v_{i+4}$ ). Además, se evidencia el peso relativo intra-componente: los nodos centrales 2 y 6 ( $v_2 = v_6 \approx 0.1706$ ) dominan sobre las hojas marginales 3 y 7 ( $v_3 = v_7 \approx 0.0604$ ).

#### 4.2.5 Grafo 4: Concentración en Subestructuras Cíclicas Recíprocas

El Grafo 4 ilustra cómo las estructuras con enlaces bidireccionales absorben la masa de probabilidad. Aunque el ciclo  $\{0, 1, 2\}$  retroalimenta flujo, el enlace unidireccional  $2 \rightarrow 3$  desvía una fracción irreducible hacia el par  $\{3 \rightleftharpoons 4\}$ . Al no existir caminos de retorno desde el dipolo  $\{3, 4\}$  hacia  $\{0, 1, 2\}$  a nivel de grafo determinístico, la mayor parte de la masa se transfiere y retiene en  $\{3, 4\}$ , logrando que estos dos nodos concentren conjuntamente más del 80% del PageRank global ( $v_3 = v_4 \approx 0.4014$ ).

#### 4.2.6 Grafo 5: Efecto de Fuentes Puras y Traspaso de Prestigio

El nodo 2 es una fuente pura (grado de entrada nulo, ningún nodo apunta a él). Su PageRank se sustenta exclusivamente en la probabilidad basal residual proveniente de la teletransportación aleatoria:

$$v_2 \approx \frac{1-d}{n} = \frac{0.15}{3} = 0.05 \quad (19)$$

A pesar de su bajo rango intrínseco ( $v_2 = 0.0635$ ), el enlace  $2 \rightarrow 0$  inyecta su probabilidad disponible de forma concentrada al nodo 0 ( $C_2 = 1$ ), rompiendo la simetría del dipolo  $\{0 \rightleftharpoons 1\}$  y posicionando al nodo 0 ( $v_0 = 0.4792$ ) por encima del nodo 1 ( $v_1 = 0.4573$ ).

### 4.3 Análisis Teórico: Calidad vs. Cantidad en Enlaces Entrantes

El principio fundacional de PageRank postula que el prestigio de un vértice no es una función aditiva elemental de su grado de entrada (número de enlaces entrantes), sino un equilibrio estacionario ponderado por la importancia de sus predecesores y diluido por los grados de salida de estos.

Formalmente, a partir de la relación de punto fijo  $v = Gv$ , el PageRank de una página específica  $i$  en un grafo sin sumideros satisface la descomposición analítica:

$$v_i = d \sum_{j \in \mathcal{I}(i)} \frac{v_j}{C_j} + \frac{1-d}{n} \quad (20)$$

donde  $\mathcal{I}(i) = \{j \in V : (j, i) \in E\}$  representa el conjunto de nodos adyacentes de entrada de  $i$ .

De la ecuación (20) se desprenden los dos motivos formales que explican por qué una página  $A$  con pocos enlaces entrantes ( $|\mathcal{I}(A)| \ll |\mathcal{I}(B)|$ ) puede superar holgadamente en ranking a una página  $B$  con una alta densidad de hipervínculos entrantes:

1. **Calidad Ponderada de la Fuente ( $v_j$ ):** El aporte de cada enlace entrante está acoplado linealmente al PageRank  $v_j$  del nodo que lo emite. Si la página  $A$  recibe un único enlace de un nodo central  $P$  de alta jerarquía ( $v_P \gg 0$ ), la contribución directa  $d \frac{v_P}{C_P}$  puede superar ampliamente la sumatoria agregada de múltiples enlaces recibidos por  $B$  provenientes de sitios periféricos de bajo rango ( $v_k \approx \frac{1-d}{n}$ ).
2. **Dilución por Grado de Salida ( $C_j$ ):** Cada nodo emisor distribuye su autoridad fraccionada en partes iguales  $\frac{1}{C_j}$  entre todas sus páginas de destino. Si la página emisora apunta de forma selectiva únicamente a la página  $A$  ( $C_P = 1$ ), transfiere íntegramente la fracción  $d \cdot v_P$  hacia  $A$ . Por el contrario, si las fuentes que apuntan a  $B$  son directorios masivos con cientos de enlaces salientes ( $C_k \gg 1$ ), el prestigio transmitido individualmente hacia  $B$  tiende asintóticamente a cero ( $\frac{v_k}{C_k} \rightarrow 0$ ) a pesar de poseer una elevada cantidad de enlaces.

Este comportamiento desacopla el ranking de la simple densidad local de conexiones, priorizando la estructura topológica global de la red.

## 5 Conclusiones

- **Efectividad del Modelo Estocástico Regularizado:** Se comprobó experimentalmente que la formulación de la Matriz de Google  $G = dM^* + \frac{1-d}{n}J$  resuelve de manera óptima las limitaciones intrínsecas de las cadenas de Markov en grafos arbitrarios. La sustitución de columnas nulas en nodos sumidero ( $C_j = 0$ ) por vectores equiprobables garantiza la conservación de la medida de probabilidad, mientras que la perturbación uniforme inducida por el factor de amortiguamiento ( $d = 0.85$ ) transforma el sistema en irreducible y aperiódico. Esto asegura la existencia y unicidad del estado estacionario bajo el Teorema de Perron-Frobenius, incluso en escenarios extremos como grafos desconectados (Grafo 3).
- **Eficiencia y Robustez del Método de las Potencias:** El Método de las Potencias demostró ser un esquema numérico estable y convergente para todas las topologías evaluadas, alcanzando la tolerancia estricta de  $\|v^{(k+1)} - v^{(k)}\|_1 < 10^{-8}$  en menos de 70 iteraciones. La tasa asintótica de convergencia observada se ajusta rigurosamente a la cota geométrica teórica gobernada por la brecha espectral ( $|\lambda_2| \leq d = 0.85$ ), evidenciando por qué este método iterativo es el estándar en problemas de gran escala frente al cómputo algebraico directo del polinomio característico.
- **Influencia de la Topología en la Distribución de Prestigio:** Los experimentos en la red propuesta de 6 nodos y los grafos de prueba permitieron constatar que los ciclos cerrados y los dipolos recíprocos actúan como trampas estocásticas que recirculan y retienen altos porcentajes de masa de probabilidad (más del 80% conjunto en el par  $\{3 \rightleftharpoons 4\}$  del Grafo 4). Asimismo, se verificó que los nodos sumidero (como el vértice 5 en la red propuesta o el nodo 2 en el Grafo 1) se comportan como atractores naturales que concentran la mayor fracción individual del vector de rangos antes de su redistribución aleatoria.
- **Desacoplamiento entre Cantidad y Calidad de Enlaces:** El análisis formal confirma que el algoritmo PageRank trasciende el conteo elemental de grados de entrada. Una página con pocos hipervínculos entrantes puede alcanzar un ranking significativamente superior al de un nodo con múltiples conexiones entrantes si la fuente emisora posee un PageRank intrínseco elevado ( $v_j$ ) y concentra su flujo a través de un grado de salida bajo ( $C_j = 1$ ). Este principio ratifica la premisa fundacional del algoritmo: la relevancia transferida depende de la jerarquía de las fuentes y del grado de dilución del flujo en la red global.